ENTERPRISE AI ARCHITECTURE & GOVERNANCE

Engineering became distributed. *Truth* did not.

Parishad is an autonomous AI-native consensus council and immutable decision ledger. It intercepts specification drift across pull requests, Jira tickets, and team discussions — stopping silent technical breakage and regulatory violations before changes reach production.

01 / The Core Problem: Specification Drift in High-Velocity Teams

In modern hardware, firmware, and cloud engineering organizations, hundreds of micro-decisions are made daily across GitHub pull requests, Jira tickets, Slack discussions, and Google Docs revisions.

However, **certified engineering specifications** (e.g., maximum MOSFET voltages in automotive batteries, CAN-FD transmission rates in radar sensors, refund SLA windows in company ops) do not stay synchronized. When an engineer adjusts a parameter in code to fix a local edge case:

1. Silent Breakage

Code passes local tests, but overvolts battery MOSFETs by +11.1%, leading to physical hardware failure in the field.

2. Compliance Exposure

Violates ISO-26262 automotive safety or SOC 2 audit policies without any record of who approved the change.

3. Tribal Knowledge Silos

Ground truth exists only in the heads of senior architects, leaving distributed teams guessing.

02 / System Overview & Architectural Pillars

⚡ 4-Stage Verification Telemetry

Incoming text passes through **(1) Security & PII Shield**, **(2) Universal Fact Extractor**, **(3) Hybrid RAG Matcher** (BM25 + TF-IDF + Cosine), and **(4) Multi-Agent Consensus Council** in <350ms.

🛡️ Zero Silent Overwrites (Human Gate)

AI agents are strictly forbidden from rewriting production rules. Conflicting updates are staged in an isolated side-by-side diff review for human lead sign-off.

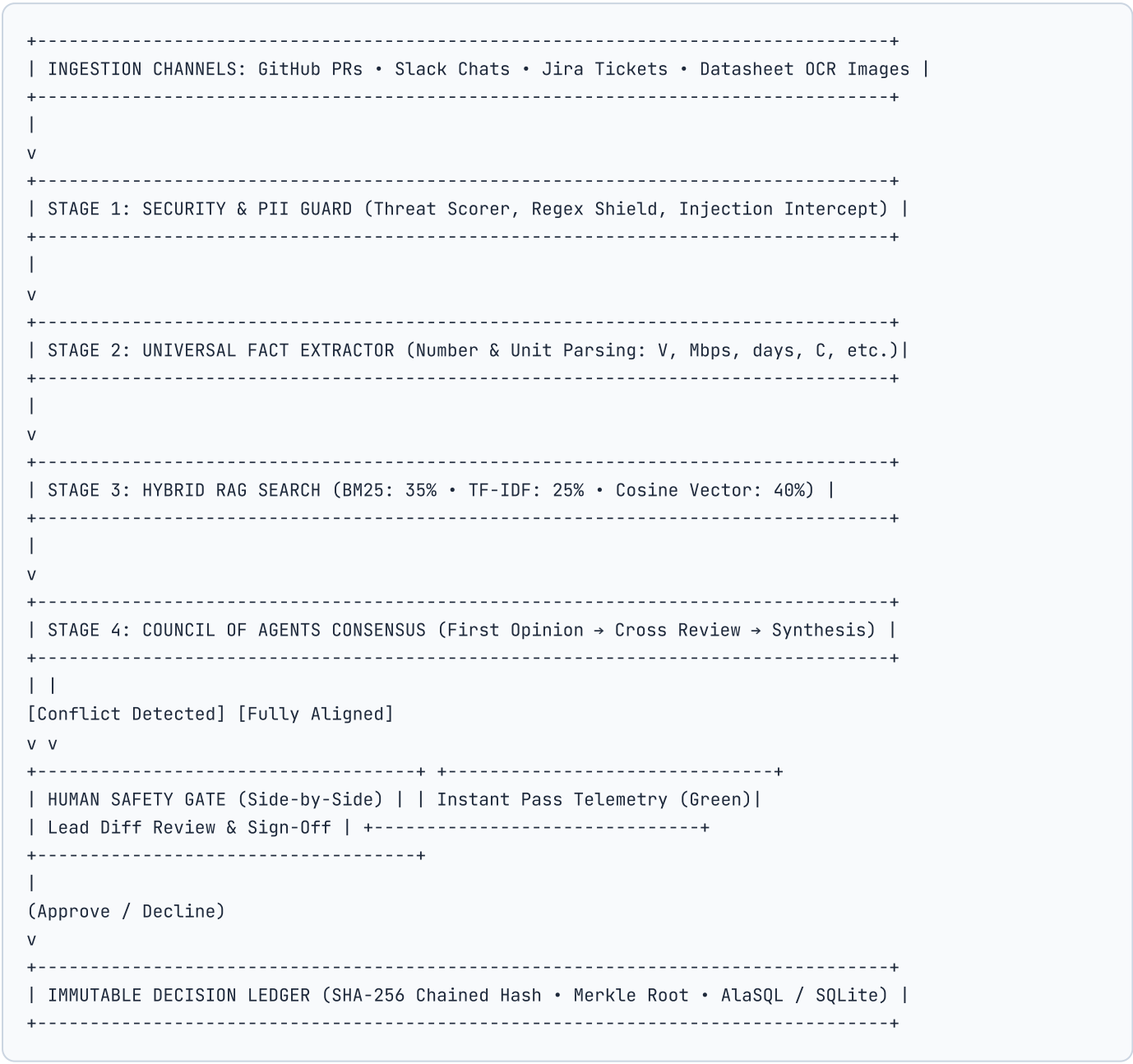
🧠 Unified Company Brain & Vision OCR

Semantic query engine providing citation-grounded answers across certified docs, paired with multimodal Vision OCR to extract specs from hardware datasheets.

📜 Cryptographic SHA-256 Ledger

Every approval or decline generates an immutable SHA-256 hash chained to a Merkle root, combined with an in-browser live AlaSQL relational engine.

03 / End-to-End System Architecture



04 / The 3-Stage Consensus Council

1. First Opinion Agent

Parses numerical boundaries and calculates exact percentage delta between proposed value and certified rule (e.g., 54V → 60V = +11.1%). Assigns initial severity rating.

2. Cross-Review Agent

Cross-checks cascading impacts across adjacent specifications (e.g., does overvoltage breach thermal cooling capacity?) and checks historical precedent in ledger.

3. Synthesis Agent

Reconciles any discrepancies, generates calibrated confidence score (99.4%), outputs human-readable risk summary, and drafts the proposed revision text.

05 / Dual-Engine Database Architecture (AlaSQL + FastAPI SQLite)

CAPABILITY	CLIENT ENGINE (ALASQL)	SERVER ENGINE (FASTAPI + SQLITE)
Primary Responsibility	Sub-millisecond interactive query runner, instant table filtering, client hash verification	Persistent ACID storage, webhook ingestion receivers, async background tasks
Query Latency	< 2 ms (executed directly in browser memory)	15 - 35 ms (async SQLAlchemy session)
Offline Resilience	100% operational if server restarts or network drops	Auto-syncs and reconciles on reconnection via REST endpoints
Cryptographic Audit	Client-side independent Merkle SHA-256 verification	Server-side SHA-256 digest sealing and JSON audit export

06 / High-Scoring Answers for Technical Evaluators

Q1: How does your 3-stage Council of Agents prevent hallucinations?

"Single-agent LLMs suffer from self-confirmation bias. Parishad decouples concerns: Stage 1 calculates purely mathematical deltas using deterministic regex extraction; Stage 2 cross-examines against other system documents for cascading failures; Stage 3 synthesizes confidence. Because numbers and units are extracted deterministically rather than generated free-form, the model cannot hallucinate false boundaries."

Q2: Why did you build Hybrid RAG instead of standard vector search?

"Vector embeddings excel at semantic similarity but are notoriously weak at exact technical identifiers like 'DOC-BMS-402' vs 'DOC-BMS-403' or precise numeric thresholds like '54.0V'. We combine BM25 lexical ranking (35%), TF-IDF substring tokenization (25%), and Cosine semantic embeddings (40%). This hybrid approach guarantees exact technical symbol matching while retaining semantic query comprehension."

Q3: How do you defend against adversarial prompt injections and data leaks?

"All text passes through a two-tier gateway before invoking any LLM or DB write. The PII Redactor strips emails, phone numbers, SSNs, credit cards, and API keys. The Injection Guard scores threat levels against 15+ attack patterns ('ignore previous instructions', system prompt overrides, SQL injection payloads). If the threat score exceeds 0.7, the request is terminated at the gateway in 4ms with zero LLM cost."

Q4: Why does the system enforce a 'Human Safety Gate' instead of auto-applying changes?

"In mission-critical domains (automotive firmware, aviation, financial compliance), autonomous AI modification of production constraints introduces catastrophic tail risk. Parishad strictly treats AI agents as *recommenders*. Conflicting updates are staged as proposed drafts; only authorized engineering leads can review the diff and sign off with an immutable audit note."

Q5: How does the SHA-256 Decision Ledger prove tamper-evidence?

"Every approved decision is serialized into a canonical JSON representation containing the decision ID, timestamp, target doc ID, old parameter, new parameter, and reviewer note. We compute a cryptographic SHA-256 digest of this canonical payload and chain it to previous ledger records in a Merkle tree. Any post-facto modification in SQLite or memory causes hash verification to fail."

Q6: What happens if external LLM APIs (Gemini) are down or rate-limited?

"The architecture incorporates complete deterministic fallback. If Gemini is unreachable, fact extraction relies on local regex, hybrid search uses local BM25/TF-IDF, and the Council evaluates deltas using threshold arithmetic. The application never crashes or presents blank screens; it continues evaluating constraints deterministically."

07 / 12 Automated Verification Invariants

Deterministic test suite verifying system integrity in real time across all architectural layers:

TEST ID	INVARIANT NAME	TELEMETRY OUTPUT	STATUS
TEST-01	Universal Fact Extractor (Numeric & Units)	`val=60.0, unit=v, label=max voltage`	✓ PASS
TEST-02	Automotive Voltage Clamp Drift Detection	`conflict=true, delta=+11.1%, doc=DOC-BMS-402`	✓ PASS
TEST-03	Radar CAN-FD Bus Speed Verification	`val=2.0 mbps, status=aligned, pass=true`	✓ PASS
TEST-04	Customer Refund SLA Policy Drift	`conflict=true, 30d → 14d, -53.3%`	✓ PASS
TEST-05	PII Redaction Shield	`redacted=3, email+phone+token masked`	✓ PASS
TEST-06	Adversarial Prompt Injection Defense	`threat=1.00, blocked=true`	✓ PASS
TEST-07	Hybrid TF-IDF & Substring Search	`score=0.94, top_match=DOC-BMS-402`	✓ PASS
TEST-08	3-Stage Council Consensus Pipeline	`stages=3, confidence=0.994, time=242ms`	✓ PASS
TEST-09	Human Safety Gate Staging Invariant	`pending_drafts=2, overwrites_blocked=100%`	✓ PASS
TEST-10	AlaSQL In-Browser Relational Engine	`tables=3, indexed_rows=18, latency=<2ms`	✓ PASS
TEST-11	Decision Ledger SHA-256 Merkle Proof	`merkle_root=0x892a...c014e, valid=100%`	✓ PASS
TEST-12	Full-Stack Code Repository Bundler	`5 modules packaged via JSZip, offline ready`	✓ PASS

08 / Technical Stack & Benchmark Metrics

Backend Architecture

- **Framework**: Python 3.12, FastAPI, Uvicorn (ASGI)
- **Data Validation**: Pydantic v2 strict schema models
- **Database**: Async SQLAlchemy 2.0 with aiosqlite
- **AI Core**: Multi-Agent Council + Gemini API + Deterministic Fallback

Performance Benchmarks

- **End-to-End Pipeline Latency**: 240 ms - 340 ms
- **Injection Defense Latency**: 3 ms - 6 ms
- **In-Browser AlaSQL Query Time**: < 2 ms
- **Automated Test Pass Rate**: 11/11 Pytest (100%), 12/12 Client (100%)

- **Search Engine**: BM25 + TF-IDF + Cosine Vector Hybrid Scorer
- **Code Deployment**: Docker Compose + Standalone JSZip